

Laboratorium 1: Podstawy Flask – Routing i Widoki

Aplikacje Internetowe

1 Cel zajęć

Celem zajęć jest zapoznanie się z mikro-frameworkiem Flask, konfiguracją środowiska pracy oraz tworzeniem podstawowych tras (routes) i statycznych widoków HTML.

2em

Tutorial i pomoc: <https://flask.palletsprojects.com/en/stable/quickstart/#a-minimal-application>

2 Zadanie 1: Konfiguracja środowiska (10-15 min)

Stwórz katalog projektu, wirtualne środowisko i zainstaluj bibliotekę Flask.

```
1 mkdir flask_app
2 cd flask_app
3 python -m venv venv
4 # Aktywacja (Windows):
5 venv\Scripts\activate
6 # Aktywacja (Linux/macOS):
7 source venv/bin/activate
8
9 pip install flask
```

Listing 1: Komendy terminala

3 Zadanie 2: Minimalna aplikacja (15 min)

Stwórz plik `app.py` i zaimplementuj najprostszą aplikację zwracającą tekst bezpośrednio w przeglądarce.

Wskazówka: Pamiętaj o imporcie klasy `Flask`. Funkcja musi zwracać string, który Flask zamieni na odpowiedź HTTP.

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     return "<p>Witaj w aplikacji Flask!</p>"
```

Listing 2: `app.py` - Minimalna aplikacja

Uruchomienie: W terminalu wpisz `flask run -debug`.

4 Zadanie 3: Tworzenie tras i zmiennych w URL (20 min)

Dodaj dwie nowe trasy:

1. `/o-nas` – zwracającą krótki opis projektu.
2. `/user/<name>` – przyjmującą parametr z adresu URL i wyświetlającą powitanie dla konkretnego użytkownika.

Wskazówka: Użyj dekoratora `@app.route('/path/<variable>')`.

```
1 @app.route("/user/<name>")
2 def show_user(name):
3     return f"Witaj, {name}!"
```

Listing 3: Przykład obsługi zmiennych w URL

5 Zadanie 4: Renderowanie szablonów HTML (15 min)

Zamiast zwracać kod HTML jako string w Pythonie, użyjemy silnika **Jinja2**.

1. Stwórz folder `templates` w katalogu projektu. 2. W środku stwórz plik `index.html`. 3. Zmodyfikuj funkcję `hello_world`, aby korzystała z `render_template`.

Wskazówka: Nie zapomnij zaimportować `render_template` z pakietu `flask`.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Strona główna</title>
5 </head>
6 <body>
7     <h1>To jest widok wygenerowany z pliku HTML!</h1>
8     <p>Praca z szablonami jest znacznie wygodniejsza.</p>
9 </body>
10 </html>
```

Listing 4: templates/index.html

```
1 from flask import Flask, render_template
2
3 @app.route("/")
4 def hello_world():
5     return render_template("index.html")
```

Listing 5: Zmiana w app.py

6 Podsumowanie

Po wykonaniu zadań student powinien posiadać strukturę plików:

- `venv/` (środowisko)
- `templates/index.html` (widok)
- `app.py` (logika i routing)

7 Zadanie Główne: Projekt "Moje Pasje" (opracowanie struktury)

Wykorzystując zdobytą wiedzę, rozpocznij budowę swojej pierwszej aplikacji tematycznej. Celem jest stworzenie szkieletu strony o Twoich zainteresowaniach.

Wymagania:

- Strona główna (/) z powitaniem i krótkim menu (tekstowym).
- Co najmniej dwie podstrony tematyczne (np. /hobby, /galeria, /kontakt).
- Każda podstrona musi być osobnym plikiem HTML w folderze `templates`.
- Wykorzystanie nawigacji: każda strona powinna zawierać proste linki HTML pozwalające przejść do pozostałych sekcji.

Wskazówka: Aby stworzyć link do innej podstrony w HTML, użyj znacznika `<a href="/nazwa-trasy">Opis</a>`.

```
1 @app.route("/")
2 def index():
3     return render_template("index.html")
4
5 @app.route("/hobby")
6 def hobby():
7     return render_template("hobby.html")
8
9 @app.route("/kontakt")
10 def kontakt():
11     return render_template("kontakt.html")
```

Listing 6: Przykładowa struktura app.py dla projektu

```
1 <nav>
2     <a href="/">Start</a> |
3     <a href="/hobby">Moje Hobby</a> |
4     <a href="/kontakt">Kontakt</a>
5 </nav>
6 <hr>
7 <h1>Witamy na stronie o moich pasjach!</h1>
```

Listing 7: Przykład nawigacji w pliku HTML